

A metaheuristic–exact hybrid for the *complete* efficient set of a multi-objective integer program, with a linear fractional decision criterion

Abstract

Let $D = \{x \in \mathbb{Z}_+^n : Ax \leq b\}$ be bounded and let $Z_1, \dots, Z_p$ be criteria to maximise, each a ratio of affine forms with a strictly positive denominator on D . We address two questions about the efficient set $E(P_D)$: producing *all* of it, and maximising a further linear fractional criterion Φ over it.

Existing methods for the second question work in *decision* space, truncating D with Sylva–Crema cuts. Each cut is a disjunction and costs p binary variables and $p + 1$ big- M rows, so every sub-problem is strictly larger than the last. We keep the unexplored part of the problem in *criterion* space instead, as a list of boxes: truncation becomes a box subtraction, every sub-problem is D plus a handful of ordinary linear rows, and the model never grows.

Three results follow. First, a characterisation that makes the complete efficient set cheap: **if v is non-dominated then every $x \in D$ with $Z(x) = v$ is efficient**, so $E(P_D)$ is exactly the union of the slices of the non-dominated set and *no efficiency test is paid on any slice point*. Second, a slice is D plus p linear equations, for fractional criteria as much as for linear ones; solving those equations eliminates r variables before enumeration begins and is worth a median $49\times$ over scanning with them as pruning rows. Third, a hybridisation that pays: a Pareto local search supplies verified efficient points which are placed in the box list without a probe. Over 150 instances it reaches the whole of $E(P_D)$ at $1.31\times$ the pure exact method, faster on 139, with identical answers throughout.

The hybridisation only pays when the seeds are handed over in the order the exact search’s own probe would meet them; in the order the archive produces them the same hybrid is *slower*. We also report, at the same length, four changes that remove a quarter of the sub-problems and buy nothing, and a single-program completeness certificate that is sound and $4.5\times$ slower, because the negative results delimit where hybridisation can help at all.

1 Problem and notation

$$(P_D) \quad \text{“max”} \quad Z_k(x) = \frac{c_k^\top x + a_k}{d_k^\top x + b_k}, \quad k = 1, \dots, p, \quad x \in D, \quad (1)$$

with $D = \{x \in \mathbb{Z}_+^n : Ax \leq b\}$ non-empty and bounded and every denominator $D_k(x) = d_k^\top x + b_k$ strictly positive on D . A point $x \in D$ is *efficient* when no $y \in D$ satisfies $Z(y) \geq Z(x)$ with at least one strict inequality; $E(P_D)$ denotes the set of efficient points and $F = Z(E(P_D))$ the non-dominated set. Linear criteria are the degenerate case $d_k = 0, b_k = 1$.

Given a decision maker’s criterion $\Phi(x) = (U^\top x + \alpha)/(V^\top x + \beta)$,

$$(P_E) \quad \max \Phi(x) \quad \text{s.t.} \quad x \in E(P_D). \quad (2)$$

$E(P_D)$ is a union of faces of D : non-convex, with no explicit description.

Throughout, the data of each criterion is scaled to integers (numerator and denominator by the same factor, leaving the ratio untouched). That makes the quantity

$$e_k(x; \bar{x}) = D_k(\bar{x}) D_k(x) (Z_k(x) - Z_k(\bar{x})) \quad (3)$$

linear in x and integer-valued, so that $Z_k(x) > Z_k(\bar{x}) \iff e_k(x; \bar{x}) \geq 1$ exactly — no minimal step has to be estimated, which is what makes the transposition from linear to fractional criteria faithful rather than approximate.

Remark 1. Φ is a function of x and **not** of $Z(x)$. Two efficient points with the same criterion vector can have different Φ . Every criterion-space construction below has to account for this, and it is the reason the transfer of criterion-space search from linear to fractional objectives is not immediate.

2 Where the cost of the decision-space method comes from

One iteration of the published approach maximises Φ over a truncated region D_l , tests the maximiser for efficiency, banks the best Φ on the criterion slice about to be destroyed, and truncates: delete $\{x : Z(x) \leq Z(\tilde{x})\}$.

Keeping the complement means keeping the x for which *some* criterion strictly improves on $\tilde{x}$. That is a disjunction, not a system of linear inequalities, and it is modelled with one binary per criterion:

$$e_k(x; \tilde{x}) \geq 1 \cdot y_k + M_k(1 - y_k), \quad k = 1, \dots, p, \quad \sum_{k=1}^p y_k \geq 1, \quad (4)$$

with M_k a lower bound on e_k over the region. **Every cut therefore adds p binaries and $p+1$ rows**, and the late sub-problems cost far more than the early ones. This fixes the cost model used throughout:

$$\text{total cost} \approx (\text{number of sub-problems}) \times (\text{cost of one, which grows}).$$

3 The same truncation in criterion space

Keep the unexplored region as a list of *boxes* instead. Deleting $\{Z \leq Z(a)\}$ from a box is a box subtraction, and it has a disjoint p -way decomposition:

$$\text{child } k : \quad e_k(x; a) \geq 1 \quad \text{and} \quad e_j(x; a) \leq 0 \quad \text{for } j < k. \quad (5)$$

Child k asks that Z_k strictly improve on a while no earlier criterion does. The children are disjoint, they cover the box minus $\{Z \leq Z(a)\}$, and — this is the point — each is D plus a handful of ordinary linear rows. **The model never grows.** A box is written with the integer-valued rows (3) rather than with numeric bounds on Z_k , which is what carries the construction to fractional criteria unchanged.

Each box also carries, for nothing, an interval per criterion: child k raises lo_k to $Z_k(a)$ and lowers hi_j to $Z_j(a)$ for $j < k$. We use it in Section 7.

4 The complete efficient set

4.1 Enumerating the non-dominated set

The enumeration keeps a list of boxes covering the unexplored region. It pops a box, *probes* it by maximising a fixed strictly positive direction over D intersected with the box, repairs the maximiser to an efficient point a , records $Z(a)$, and splits the box around a by (5).

Proposition 1. *The enumeration terminates and records every non-dominated vector exactly once.*

Proof. Termination. The probe returns a point x of D inside the box, and the efficient point a it is repaired to dominates it, so $Z(x) \leq Z(a)$ and the split around a removes at least $Z(x)$ from that box. The unexplored region shrinks on every pass, and the non-dominated set of a bounded integer program is finite.

Completeness. A non-dominated v leaves the region only through a split around some centre a with $v \leq Z(a)$. If $v \neq Z(a)$ that says a dominates v , which no non-dominated vector admits. So v leaves only at the moment it is recorded; the loop ends only when the region is empty, so every v is recorded, exactly once since centres are de-duplicated by criterion vector. $\square$

4.2 From the non-dominated set to the efficient set

Completeness of F is not completeness of $E(P_D)$: one vector can be attained by several efficient points and the enumeration keeps one per vector. The gap closes on one observation.

Theorem 1. *If v is non-dominated, then **every** $x \in D$ with $Z(x) = v$ is efficient.*

Proof. Suppose some $y \in D$ dominated such an x . Then $Z(y) \geq Z(x) = v$ with at least one strict inequality, which says v is dominated — against the hypothesis. $\square$

Corollary 1. $E(P_D) = \bigcup_{v \in F} \{x \in D : Z(x) = v\}$, and **no efficiency test is paid on any slice point.**

The dearest operation in the whole method — the efficiency test — is simply not invoked on the slices. What remains is to enumerate each slice.

4.3 A slice is D plus p linear equations

Fixing $Z(x) = v$ is linear in x *even for fractional criteria*: since $D_k(x) > 0$ on D , the equation $(c_k^\top x + a_k)/(d_k^\top x + b_k) = v_k$ clears to

$$(c_k - v_k d_k)^\top x = v_k b_k - a_k, \quad (6)$$

with no sign condition and no case split. A slice therefore carries no binary, no big- M and nothing that grows from one slice to the next.

These are equations, so *solve* them rather than use them to prune. Gaussian elimination in exact rationals expresses r of the variables as linear functions of the rest; substituting into every row of the model leaves an equivalent system in $n - r$ unknowns, on which the enumeration runs. Two rows per eliminated variable keep it inside its own bounds. At each leaf the eliminated variables are rebuilt and the one condition that is *not* linear — that they come out integer — is checked there. An inconsistent system settles the slice as empty with no enumeration at all. The search space $(ub + 1)^n$ becomes $(ub + 1)^{n-r}$.

Table 1: Pruning on the slice equations against solving them. Fourteen instances, $n = 7 \dots 12$; identical point sets throughout.

instance	prune	eliminate	ratio
$n = 9, p = 3$	29.71 s	0.31 s	96.7×
$n = 10, p = 5$	119.47 s	0.92 s	129.9×
$n = 12, p = 3$	796.84 s	8.34 s	95.5×
median of all fourteen			49.2×

Every coefficient is a rational and the elimination is exact, so this changes the order of the enumeration and nothing about its output.

4.4 Pairing each vector with a maximiser of Φ

By Remark 1 the front alone does not answer (P_E) . Each vector is therefore paired with the maximiser of Φ *on its slice*, at one extra integer program per vector. That pairing has a consequence worth stating: **the largest value on the front is the optimum of (P_E)** , so the enumeration is a second, independent exact method for (P_E) and serves as a cross-check on the first.

5 The hybridisation

5.1 What a seed can feed here, and what it cannot

A seed for an *optimisation* search feeds one mechanism: the bound. A search that already knows a good value prunes boxes whose bound cannot beat it. **The enumeration has no bound and no incumbent** — it must exhaust every box — so a seed of that kind is worth exactly nothing.

It has a different mechanism. Every box costs a probe, an efficiency test and sometimes a repair, all spent to find *a vector*; a vector already known needs none of them. Seeding here is not “start from a better value” but *do not pay to discover what you already know*.

Proposition 2. *Let $S \subseteq E(P_D)$ be a set of efficient points. Placing each $a \in S$ in the box list by locating the unique box containing it and splitting that box around a by (5), then running the enumeration, returns the same F and the same $E(P_D)$, still complete by Proposition 1.*

Proof. The split (5) is disjoint and covers the box minus $\{Z \leq Z(a)\}$, so a point whose vector has not been removed lies in exactly one child; the location is therefore well defined, and a seed whose vector has already been removed is dominated by an earlier one and is discarded. The operation performed on the list is exactly the one the enumeration performs when its probe returns a : same centre, same rows, same children. The proof of Proposition 1 appeals only to the centres being feasible and to the region shrinking, both of which hold verbatim. Efficiency of the seeds is what makes the *recorded* vectors non-dominated, and is required for that reason alone. $\square$

So the heuristic decides how much of the front has to be *discovered*, and nothing about what is returned.

5.2 Which heuristic, and why the verification is free

Two sources of efficient points were compared.

The **augmented weighted Tchebychev program** is the natural candidate: its optima are efficient by construction, for any strictly positive weights and any positive augmentation, so its output needs no verification. A **Pareto local search** with an archive proves nothing, so every member has to be put through the exact efficiency test.

That verification looks like the deciding cost and is not, because of an accounting that holds only here: **the enumeration was going to pay the same efficiency test anyway**. Every probe a seed removes was going to be followed by one. The verification cancels, and the probe is the gain. What a seed really costs is its share of the walk.

Table 2: The two sources, per efficient point.

	cost of one point	verification	front covered
Tchebychev scalarisation	20.90 ms	none needed	33–100%
Pareto walk (whole archive)	0.011 s total	one test each	95% median

And a scalarisation is not cheaper than the probe it replaces: one probe costs 9.48 ms and 0 branch & bound nodes, one scalarisation 20.90 ms and 21 nodes, cheaper on only 7 of 40 instances. The probe maximises a linear direction over D plus ordinary rows and usually needs no branching at

all; the scalarisation minimises a *maximum*, at the price of a continuous variable and p rows, and it branches.

Consistently with that arithmetic, seeding from the generator *loses*, and loses *more the more of the front it seeds*: with a wide weight grid it covers 100% of the front and runs at $2.61\times$ the unseeded time.

5.3 The order in which seeds are handed over

Placing k seeds was expected to remove k probes. In the median it does ($+1.00$ over 152 instances), but on 5 of those 152 the seeded run spent *more*: the pre-split imposes a box structure the search would not have chosen, and a box it would have absorbed into a larger one now exists separately and must be probed.

The probe maximises a fixed direction, so the search splits first around whatever scores highest on it. Sorting the seeds by that same direction reproduces the structure the search would have built.

Table 3: Probes against the unseeded enumeration, by the order the seeds arrive. Twelve instances, $n = 6 \dots 10$.

order	probes	raised the count on
archive order	$0.89\times$	4 of 12
probe order	$0.70\times$	none
reversed	$0.94\times$	5 of 12

The reversed arm is what makes the direction causal rather than a coincidence of this sample. In archive order the hybrid is *slower* than the pure exact method, at $0.81\times$; in probe order it is $1.33\times$ faster.

6 Computational results

All arithmetic is exact and rational, with no floating point anywhere, so every comparison is between two exactly solved problems. Instances are drawn from a documented pseudo-random family; the statistic is the *median of the per-instance ratios* with the interquartile range, never a ratio of totals, which would be decided by the single heaviest instance.

Table 4: The hybrid against the pure exact methods. Thirty instances at each of five configurations, $n = 6 \dots 9$, $p = 3, 4$.

	median	IQR	faster on
front enumeration	$1.33\times$	[1.20, 1.51]	139/150
$E(P_D)$ complete	$1.31\times$	[1.17, 1.47]	139/150
probes	$0.72\times$	[0.67, 0.75]	<i>never worse</i>

The answer is identical on every one of the 150 — same front, same $E(P_D)$, same optimum of (P_E) — which the experiment asserts rather than assumes.

For context, the underlying substitution of criterion space for decision space is worth the following on (P_E) itself, thirty instances per row at $n = 6$:

The last row is the cost model of Section 2 read off the measurement: at $p = 6$ the two methods solve *the same number* of sub-programs and one still takes $14.57\times$ as long, so the whole margin lies in the cost of a single sub-problem, whose branch & bound explores $5.37\times$ the nodes because it carries p binaries and $p + 1$ big- M rows per cut.

Table 5: Decision space against criterion space on (P_E) .

p	ratio (median)	IQR	sub-programs	b&b nodes
2	1.52×	[1.25, 1.91]	1.72×	1.28×
3	2.69×	[1.87, 3.72]	1.40×	1.79×
4	4.99×	[2.39, 11.62]	1.19×	2.68×
5	7.93×	[3.61, 19.98]	1.17×	3.16×
6	14.57×	[7.90, 25.17]	1.02×	5.37×

Measurement discipline. The machine used has a noise floor of roughly $\pm 30\%$ per instance even at per-instance minima over five repeats, so a small effect cannot be resolved in seconds and no number of repeats fixes that. Deterministic work counts — sub-programs, branch & bound nodes, simplex pivots — are reported next to the times and are identical on every run and every machine. Runs stopped by a time budget are reported as censored and excluded from medians rather than counted as their budget.

7 Negative results, and a wall

We report these at the same length as the positive ones, because they delimit where hybridisation can help at all.

7.1 Four changes that remove a quarter of the sub-problems and buy nothing

Table 6: Four attempts on the sub-problem count.

change	sub-problems removed		cost
early exit on a hopeless bound	25%	no measurable time	
free contradiction filter	24.8%	0.98×	
range filter (ideal, anti-ideal)	+11.3%	1.03×	
dominated-corner filter	8–23%	1.077×	

All four are exact and one-sided: they discard only boxes that provably hold nothing, so they can change the cost and not the answer. Only the second is kept, and it is worth 2%. After four attempts the reason is a property rather than an accident:

In this search, removing sub-problems does not remove time, because the sub-problems that can be removed cheaply are the ones that were already cheap.

73% of all boxes are empty, and an empty box is refuted by the simplex almost at once: every probe in these families costs *zero* branch & bound nodes, because the relaxation is integral or infeasible at the root. A filter that catches empty boxes is catching the cheapest work in the search. The only change reported here that goes round this wall is Section 4.3, which attacks an *exponent* rather than a count.

7.2 A single-program completeness certificate

After the walk seeds 95% of the front, 93% of the remaining boxes are empty and the enumeration is almost purely a *proof of emptiness*. But “is there a feasible point not dominated by any of the k known ones?” is **one** question, and the box list answers it by splitting it into hundreds.

Formulation (4) answers it in one integer program with k cuts, built once and never iterated — which removes exactly the growth that makes that formulation expensive in Section 2.

It is sound: 13 of 30 instances were proved complete by a single program, with no wrong verdict. It is also $4.5\times$ slower in the median and $100\times$ slower at worst (239 s against 1.95 s), because k cuts mean kp binaries in one model — 88 of them at $k = 22$, $p = 4$.

This answers a question wider than an optimisation. Criterion space is the better *search* because the model never grows; the certificate tests whether it is also the better *proof*, where there is no iteration and no growth at all. **It is, by $4.5\times$:** answering many cheap questions beats answering one question with 88 binaries.

7.3 What the failures have in common

Four hybridisations were attempted in this line of work. Handing the exact search the unfinished work of the decision-space method ties it. Seeding the optimisation search from the Tchebychev program never pays, because that program steers in criterion space relative to the ideal point and never looks at Φ . Seeding the *enumeration* from the same program loses, because a scalarisation costs more than the probe it replaces. Only the Pareto walk pays, and only in probe order. The rule that survives all four:

A hybridisation pays only where the exact method has a mechanism the first phase can feed **more cheaply than the exact method feeds it itself**.

The first three fail the second half of that test. The fourth passes it because its verification is a test the exact method had already budgeted for.

8 Conclusion

Keeping the unexplored region in criterion space removes the growth that dominates decision-space methods for (P_E) , and the same substitution carries to complete enumeration of the non-dominated set. Theorem 1 then turns that enumeration into the complete efficient set at no efficiency test at all, and solving the slice equations rather than pruning on them makes the remaining enumeration $49\times$ cheaper. On top of that, a Pareto local search whose verified archive is handed over in probe order reaches all of $E(P_D)$ at $1.31\times$ the pure exact method over 150 instances, with identical answers.

The negative results are the boundary of the positive one. A seed is worth what the exact method can do with it, and in an enumeration — which has no bound to feed — it is worth exactly the discovery it replaces, priced against what discovery would have cost.

Limitations. The instances are drawn from one pseudo-random family with $n \leq 12$ and $p \leq 6$. The *directions* reported here are explained by mechanisms that are themselves measured — sub-program counts, node counts, pivot counts — but the *magnitudes* are properties of that family and should not be read as expected values elsewhere. A full bibliographic review, including sources behind paywalls, remains to be done before any claim of novelty is made: what is stated above is what we measured, not what we believe to be new.

Reproducibility. The implementation is pure Python with no third-party dependency, computes in exact rational arithmetic throughout, and carries 92 tests. Several check *properties* rather than answers — the split is a partition, the metaheuristic never hands over an inefficient point, a budgeted run never claims completeness — since a method that silently lost efficient points would still agree with the optimum on favourable instances. Per-instance records of every experiment are stored so that each table can be rebuilt and re-checked without re-running it.